

Contents

guide CUDA C++ Programming Guide	1
0. 这篇导读应该怎么用	1
1. CUDA 的大 story: host 和 device 的分工	2
2. GPU 硬件模型: SM、GPC、memory	3
3. Grid、Block、Thread: 最核心的层级	3
4. Block 调度: 正确性不能跨 block 等待	4
5. Warp 和 SIMT	4
6. Thread 线性化: x 最快	5
7. Memory spaces: scope、lifetime、location	5
8. Global memory coalescing	6
9. Shared memory: 快, 但需要同步	6
10. Shared memory bank conflicts	7
11. Occupancy: active warps / maximum warps	7
12. Tiled GEMM: 为什么 CUDA 喜欢 tile	8
13. Reductions and warp primitives	9
14. Streams: CUDA 的异步 work queue	9
15. Events and synchronization	10
16. CUDA Graphs: 减少重复提交开销	10
17. Thread block clusters and distributed shared memory	11
18. Tile programming: 新接口, 不是替代 SIMT	11
19. Unified Memory 和显式内存管理	12
20. 本仓库代码怎么对应 Guide	12
21. 和 LLM 系统的连接	13
22. 用 AI agent 学 CUDA 的正确方式	14
23. 新手复习问题	14
24. 一句话收束	15

guide_CUDA C++ Programming Guide

原文: CUDA Programming Guide, Release 13.3

本地原文 PDF: [learning/cuda-essentials/paper/01_cuda_cpp_programming_guide.pdf](#)

作者: NVIDIA Corporation

本地 PDF 日期: 2026-05-27

类型: primary document / official programming guide

0. 这篇导读应该怎么用

CUDA Programming Guide 不是普通论文, 而是官方编程模型手册。它有 600 多页, 真正阅读时不能逐段背诵。正确读法是先建立一张执行模型图:

```
host code on CPU
  allocates/copies data
  launches kernels
  synchronizes streams/events
```

```
device code on GPU
  grid
    block
      warp
        lane/thread
```

```
memory
  registers
  shared memory / L1
  L2
  global device memory
  host memory / unified memory
```

这篇导读的目标是让你读完后能做到三件事:

- 看到一个 CUDA kernel, 知道每个 thread 负责哪个数据元素。
- 看到一个访存模式, 能判断它是否 coalesced、是否有 shared memory bank conflict。
- 看到一个 kernel 很慢, 能从 occupancy、memory traffic、warp divergence、sync、stream ordering、launch overhead 这些角度诊断。

如果上一篇 Roofline 是 “先判断 memory-bound 还是 compute-bound”, CUDA Guide 就是 “把这个判断落到代码和硬件执行层级”。

1. CUDA 的大 story: host 和 device 的分工

CUDA 假设的是 heterogeneous system, 也就是 CPU 和 GPU 共存:

- CPU 以及它直接连接的内存叫 host 和 host memory。
- GPU 以及它直接连接的内存叫 device 和 device memory。
- 应用总是从 CPU 开始执行。
- CPU 通过 CUDA API 分配 device memory、拷贝数据、launch kernel、等待完成。
- GPU 执行 device code。被 launch 到 GPU 上的函数叫 kernel。

可以把一个 CUDA 程序读成:

```
host:
  prepare input
  cudaMalloc / cudaMallocManaged
  cudaMemcpy or cudaMemcpyAsync
  kernel<<<grid, block, shared_mem, stream>>>(...)
  cudaStreamSynchronize or cudaDeviceSynchronize
  read output
```

```
device:
  many threads run the same kernel body
  each thread computes an index
  each thread reads/writes its assigned data
```

新手最容易犯的错是把 GPU 当成 “更快的 CPU”。CUDA 的真正抽象不是 “调用一个函数”, 而是 “一次 launch 产生大量 threads, 这些 threads 被组织成 block/grid, 并在 SM 上调度”。

2. GPU 硬件模型: SM、GPC、memory

Guide 用一个概念模型描述 GPU:

- GPU 由多个 Streaming Multiprocessors, SMs, 组成。
- SM 可以组织成 Graphics Processing Clusters, GPCs。
- 每个 SM 有 register file、unified data cache、shared memory/L1 资源和功能单元。
- GPU 连接到 device memory。
- CPU 和 GPU 之间通过 PCIe、NVLink 或系统互连连接。

重要的是, CUDA programming model 不要求你知道所有物理细节。硬件实际 layout 可以变, 但只要你遵守编程模型, 程序正确性就不应该依赖某个具体 SM 排布。

这也是为什么手册反复强调:

不要依赖 block 的调度顺序

不要让一个 block 等另一个 block 的结果

不要利用某个 GPU 代际的未公开行为写 correctness

性能优化可以利用 warp、coalescing、shared memory、occupancy 等规律, 但正确性必须建立在 CUDA 明确承诺的语义上。

3. Grid、Block、Thread: 最核心的层级

kernel launch 会产生一个 grid。grid 由多个 thread blocks 组成。每个 block 由多个 threads 组成。

```
grid
  block 0
    thread 0
    thread 1
    ...
  block 1
    thread 0
    thread 1
    ...
```

grid 和 block 都可以是 1D、2D 或 3D。这主要是为了方便把线程映射到数据:

- vector 通常用 1D。
- matrix 通常用 2D。
- volume/stencil 可能用 3D。

线程通过内置变量知道自己是谁:

```
gridDim.x/y/z
blockDim.x/y/z
blockIdx.x/y/z
threadIdx.x/y/z
```

最经典的 1D global id:

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

对应本仓库:

```
def global_thread_id_1d(block_idx_x, thread_idx_x, block_dim_x):
    return block_idx_x * block_dim_x + thread_idx_x
```

如果 problem size 不是 block size 的整数倍，就要 guard:

```
if (i < n) {  
    C[i] = A[i] + B[i];  
}
```

这个 if (i < n) 不是可选细节。最后一个 block 往往有多余 threads，如果不 guard，就会越界写 global memory。

4. Block 调度: 正确性不能跨 block 等待

Guide 说得非常清楚:

- 一个 block 的 threads 会在同一个 SM 上执行。
- 同一个 block 内的 threads 可以高效通信和同步。
- 不同 blocks 会被调度到可用 SM 上。
- block 的调度顺序没有保证。
- grid 可以很大，SM 可以很少，blocks 会分批执行。

所以一般 CUDA kernel 必须满足:

```
blocks can execute in any order  
blocks can execute in parallel or in series  
no correctness dependency between blocks in the same grid
```

这条规则很关键。比如你不能在一个普通 kernel 中让 block 0 先写一个值，然后 block 1 等待它，再继续计算。因为 block 1 可能先被调度，也可能 block 0 和 block 1 根本不同时驻留。

如果需要跨 block 协作，通常有几种路径:

- 分成多个 kernel launch，中间由 kernel boundary 形成全局同步。
- 使用 atomics 做受控的 global memory 更新。
- 使用 cooperative groups 或 thread block clusters，遵守其额外限制。
- 改算法，让每个 block 独立生成 partial result，再下一阶段 reduce。

本仓库的 cuda_original_minimal.py 用 schedule_blocks 展示 block 会按 waves 被安排到有限 SM capacity 上，但这只是教学模拟，不代表真实硬件承诺某个顺序。

5. Warp 和 SIMT

block 内 threads 会组成 warp。一个 warp 是 32 个 threads。

Guide 的核心描述是:

```
warp executes kernel code in SIMT:  
Single Instruction, Multiple Threads
```

SIMT 和 SIMD 很像，但不完全一样:

- SIMD 暴露固定 vector width。
- SIMT 让每个 thread 有自己的 state 和 control flow。
- 同一个 warp 中 threads 可以走不同 branch。
- 如果 warp 内 threads 走不同 branch，就会出现 warp divergence。

Warp divergence 的直觉:

```
if only half lanes enter branch:
    active lanes execute branch
    inactive lanes are masked
    then lanes reconverge
```

这不会让程序自动错，但会浪费执行资源。利用率最高的情况是同一个 warp 内 threads 尽量走同一控制流。

另外，一个 block 的 thread 数最好是 32 的倍数。不是倍数也合法，但最后一个 warp 会有 unused lanes，功能单元和 memory transaction 利用率变差。

本仓库对应：

```
def warp_and_lane(linear_tid):
    return linear_tid // 32, linear_tid % 32
```

以及 warp_primitives.py:

- shfl_down_sync
- warp_reduce_sum
- ballot_sync

这些都在帮你理解 warp 是 CUDA 中非常真实的执行单位。

6. Thread 线性化: x 最快

Guide 提醒，multi-dimensional block/grid 只是 convenience，不改变性能本质。线程在 block 内会按稳定方式线性化：

```
threadIdx.x moves fastest
then threadIdx.y
then threadIdx.z
```

公式：

```
linear_tid
= threadIdx.x
+ threadIdx.y * blockDim.x
+ threadIdx.z * blockDim.x * blockDim.y
```

这个公式不仅用于索引，也影响 warp 形成。连续的 threadIdx.x 通常落在同一 warp 相邻 lanes 中。因此，想让 memory coalescing 好，通常让 threadIdx.x 对应连续内存地址。

典型错误：

```
threadIdx.x maps to row index
row-major matrix uses row * ld + col
consecutive lanes jump by ld
global write becomes uncoalesced
```

这就是手册用 matrix transpose 反复解释的核心。

7. Memory spaces: scope、lifetime、location

CUDA device memory spaces 可以按 scope/lifetime/location 记：

- global memory: grid scope, application lifetime, device memory.

- constant memory: grid scope, application lifetime, device memory.
- shared memory: block scope, kernel lifetime, SM。
- local memory: thread scope, kernel lifetime, device memory.
- registers: thread scope, kernel lifetime, SM。

新手最需要理解这几层:

register:

per thread, fastest, compiler managed

shared memory:

per block, on SM, explicit programmer-managed scratchpad

global memory:

visible to all threads, persistent, high latency, large capacity

local memory:

thread-private but often spilled into device memory, can be slow

不要被名字骗。local memory 不是 “离 thread 很近的快内存”。它是 thread-private 的 address space, 实际可能落到 device memory, 所以 register spill 会很贵。

8. Global memory coalescing

Global memory performance 是很多 CUDA kernel 的第一性能问题。

Guide 的解释是: 一个 warp 中 32 个 threads 发出 load/store 时, 硬件会把这些地址 coalesce 成若干 memory transactions。目标是让 fetched bytes 尽量都被 threads 使用。

手册举例:

- 每个 thread 读 4-byte word。
- consecutive threads 读 consecutive words。
- 32 lanes 共需要 128 bytes。
- 这些会形成高效 coalesced access。

如果 consecutive threads 访问相隔很远的地址, 最坏情况下每个 thread 都需要单独 transaction。手册中最坏例子是 32 threads 只使用 128 bytes, 但触发总共 1024 bytes traffic, 利用率只有 12.5 percent。

用一句话记:

coalescing = maximize useful bytes / transferred bytes

本仓库 coalescing.py 用 128-byte sector 的简化模型训练这个直觉:

```
coalesced_load(0)           # one sector
strided_load(0, 32)         # each lane lands in its own sector
efficiency(addresses)       # useful bytes / loaded bytes
```

真实硬件 transaction 细节会随架构变化, 但学习目标不变: 让同一个 warp 的 lanes 尽量访问 相邻或同一段内存。

9. Shared memory: 快, 但需要同步

Shared memory 是 block 内 threads 共享的 on-chip scratchpad。它比 global memory 小得多, 但通常 latency 更低、bandwidth 更高。它常用于:

- block 内交换数据。
- tiling, 提高数据复用。
- transpose, 改善 global memory coalescing。
- reduce/scans, 做中间 partial results。

因为 shared memory 是多个 threads 共享的, 所以必须考虑 race。最基本同步是:

```
__syncthreads();
```

它让 block 内所有 threads 到达同一个 barrier 后再继续。典型 shared memory pattern:

```
threads load global memory into shared memory
__syncthreads()
threads reuse shared tile
__syncthreads()
next tile
```

缺少同步会出现:

- 某些 thread 还没写完 shared memory, 另一些 thread 就开始读。
- 某些 thread 已经覆盖下一轮 tile, 另一些 thread 还在读上一轮 tile。

10. Shared memory bank conflicts

Shared memory 通常分成 32 banks。连续的 32-bit words 映射到连续 banks。

如果同一个 warp 内多个 threads 访问同一个 bank 的不同地址, 就会 bank conflict, 访问被 serialize。例外:

- 多个 threads 读同一个地址时可以 broadcast。
- 写同一个地址时由某个 thread 写入, 具体哪一个不应依赖。

手册的 matrix transpose 例子非常经典。声明:

```
__shared__ float smemArray[32][32];
```

如果 warp 中 consecutive lanes 沿 column 访问, 地址 stride 是 32 个 float, 所有 lanes 可能打到同一个 bank, 形成 32-way conflict。如果沿 row 访问, 则 consecutive lanes 访问相邻 words, 没有 conflict。

常见修复:

```
__shared__ float smemArray[32][33];
```

多加一列 padding, 让 column access 的 bank 映射错开。

本仓库 shared_memory.py 用简化模型展示:

```
stride_access(1)    # no conflict
stride_access(32)   # 31 conflicts
count_conflicts([4] * 32) # broadcast, no conflict
```

这一节和 Roofline 直接相连: shared memory tiling 和 padding 不是为了 “看起来高级”, 而是为了减少 HBM traffic 或避免 shared memory serialization。

11. Occupancy: active warps / maximum warps

Guide 定义 occupancy:

occupancy = active warps on an SM / maximum warps supported by that SM

高 occupancy 通常有助于隐藏 latency, 但不是越高越一定快。它受资源限制:

- threads per block.
- blocks per SM limit.
- registers per thread / per block.
- shared memory per block.
- max threads per SM.

手册给了两个非常适合记忆的例子, 假设一个 SM 最多 2048 resident threads:

```
kernel<<<512, 768>>>:
    each block has 768 threads
    only 2 blocks fit by thread count
    active threads = 1536
    occupancy = 75 percent
```

```
kernel<<<512, 32>>>:
    each block has 32 threads
    max 32 blocks per SM
    active threads = 1024
    occupancy = 50 percent
```

如果一个 block 用 100 KB shared memory, 而 SM 总 shared memory 约 233 KB, 那么一个 SM 最多放 2 个这样的 blocks. register 也类似。限制 register 可以提高 occupancy, 但如果导致 spill 到 local memory, 性能可能反而变差。

所以 occupancy 的正确读法是:

occupancy is a latency-hiding resource, not the final goal

本仓库 `cuda_original_minimal.py` 的 `occupancy_from_threads` 复现了这两个例子。

12. Tiled GEMM: 为什么 CUDA 喜欢 tile

Tile 是 GPU 优化的核心思想之一。对于 GEMM:

$C = A @ B$

naive 版本里, 每个 output element 由一个 thread 计算, 循环 K。问题是同一块 A/B 数据会被很多 threads 反复从 global memory 读。

tilted 版本:

```
load A tile into shared memory
load B tile into shared memory
__syncthreads()
many threads reuse those tiles
accumulate partial C
__syncthreads()
next K tile
```

这样可以用 shared memory 换取 global memory traffic 减少。gemm_tiled.py 用 Python 模拟了这个故事, 并对比 naive/tilted HBM traffic。测试里 1024 cube GEMM 的简化 traffic 下降约 32x。

这就是 CUDA + Roofline 的核心组合:

tiling reduces bytes
operational intensity rises
kernel moves right on Roofline
compute resources become more reachable

13. Reductions and warp primitives

Reduction 是 CUDA 入门后第一个真正难的模式。它不是每个 thread 独立写一个 output, 而是很多 threads 共同产生一个结果。

naive CPU:

```
sum = 0
for x in data:
    sum += x
```

CUDA 需要并行树:

```
step 1: pairwise sums
step 2: quarter active threads
step 3: eighth active threads
...
```

在 warp 内, 可以用 shuffle primitives 做 reduce, 减少 shared memory 使用和同步开销。本仓库 warp_primitives.py:

```
warp_reduce_sum(values)
```

会模拟 32 lanes 通过 shfl_down 做 5 步 reduce。

关键学习点:

- block 内同步可以用 shared memory + __syncthreads()。
- warp 内 reduce 可以用 warp primitives。
- 跨 block reduce 通常要分阶段 kernel 或 atomics。
- 浮点 reduction 的顺序改变会带来数值差异, 这在真实 GPU 上很常见。

14. Streams: CUDA 的异步 work queue

Guide 把 stream 定义成一个 work queue。你可以往 stream 里放:

- memory copy。
- kernel launch。
- event。
- callback 或其它异步任务。

同一个 stream 内:

operations execute in enqueue order

不同 streams 之间:

may overlap, depending on hardware resources and dependencies

这解释了很多 CUDA 程序的性能:

```
stream 0:
  H2D batch 0
  kernel batch 0
  D2H batch 0
```

```
stream 1:
  H2D batch 1
  kernel batch 1
  D2H batch 1
```

如果数据独立、硬件支持、host memory 是 pinned/page-locked, copy 和 compute 才可能重叠。Guide 特别提醒: `cudaMemcpyAsync` 如果使用非 pinned host memory, 可能退化为同步行为, 从而无法获得 overlap 收益。

本仓库 `cuda_original_minimal.py` 的 `stream_timeline` 展示:

```
same stream:
  next op starts after previous ends
```

```
different stream:
  op can start at time 0 if no dependency
```

15. Events and synchronization

CUDA 里同步有多个层级:

- device-level sync: 等整个 device 上的 work。
- stream-level sync: 等某个 stream 清空。
- event-level sync: 等某个标记完成。
- block-level sync: `__syncthreads()`。
- cooperative groups sync: 特定 group 的同步。

新手常见错误是用太大的同步锤子。比如每个小 kernel 后面都 `cudaDeviceSynchronize()`, 会破坏异步 overlap。更好的做法是用 stream/event 表达最小必要依赖。

也要记住 CUDA launch 和很多 runtime calls 对 host 是 asynchronous 的。也就是说, CPU 发起 kernel 后可能马上返回。错误也可能异步出现, 所以需要正确 error checking 和 同步点来定位问题。

16. CUDA Graphs: 减少重复提交开销

如果应用反复执行同一串 CUDA operations, 比如:

```
copy input
kernel 1
kernel 2
kernel 3
copy output
```

每次都从 host 单独提交这些 API calls 会有 CPU overhead。CUDA Graphs 允许把这串操作先 capture 或手动构造成 DAG, 然后 instantiate, 之后重复 launch 这个 graph。

Guide 给出的三阶段:

```
capture or create graph
instantiate graph
execute pre-instantiated graph many times
```

这对 LLM 推理尤其有意义。decode 阶段每 token 的工作可能很多小 kernel, CPU launch overhead 会变成瓶颈。CUDA Graphs、persistent kernels、fused kernels 都是减少提交/调度 开销的思路。

本仓库:

```
graph_submission_overhead_us(n_ops=10, repeats=100)
```

用一个小模型展示 repeated host launches 和 graph replay 的 overhead 差异。

17. Thread block clusters and distributed shared memory

Guide 还讲了 compute capability 9.0+ 的 thread block clusters。

普通 CUDA:

```
block is the main cooperative unit
blocks in one grid generally cannot synchronize with each other
```

clusters 增加了一层:

```
cluster = group of adjacent thread blocks
blocks in a cluster are co-scheduled in one GPC
cluster can use cooperative groups
cluster can access distributed shared memory
```

这不是入门第一天必须写的东西, 但它很重要, 因为现代 GPU 逐渐提供更强的跨 block 协作能力。学习时要分清:

- 普通 block shared memory: block scope。
- distributed shared memory: cluster scope。
- global memory + atomics: broader but slower and 更复杂。

18. Tile programming: 新接口, 不是替代 SIMT

Release 13.3 Guide 里有不少 tile programming 内容。它让 programmer 在 block/tile 层面表达操作, 由 compiler 映射到 block 内 threads。它和 SIMT 共存, 不是替代 SIMT。

可以这样理解:

SIMT:

```
programmer thinks about individual threads
manually computes indexes
manually coordinates shared memory and sync
```

tile programming:

```
programmer thinks about block-level tile operations
compiler maps tile operations to threads
```

对本仓库当前学习目标来说, 先掌握 SIMT 是基础。tile programming 可以作为 “未来 API 如何提高抽象层” 的阅读对象, 但不要跳过 thread/block/warp/memory 这些底层概念。

19. Unified Memory 和显式内存管理

Guide 同时介绍 explicit memory management 和 Unified Memory。

显式管理:

```
cudaMalloc  
cudaMemcpy  
cudaFree
```

优点是控制明确，性能行为更容易预测。缺点是代码多。

Unified Memory:

```
cudaMallocManaged
```

优点是统一地址空间，编程更容易。缺点是 page migration、prefetch、placement 可能影响性能。手册也介绍 memory advise 和 prefetch，用来帮助 runtime 把数据放到合适位置。

对性能学习来说，建议:

- 初学可以用 Unified Memory 理解功能。
- 性能实验要理解数据何时在 host/device 之间迁移。
- 大规模 LLM 训练/推理通常需要非常明确地控制 memory placement。

20. 本仓库代码怎么对应 Guide

本专题代码都是 Python mock，目标是帮助你在没有真实 CUDA 编译环境时也能练机制。

common.py:

- Grid
- Block
- Thread
- launch_config

对应 grid/block/thread 和 1D launch。

cuda_original_minimal.py:

- linear_thread_index
- global_thread_id_1d
- warp_and_lane
- schedule_blocks
- stream_timeline
- graph_submission_overhead_us
- occupancy_from_threads

对应 Guide 的核心语义。

vector_add.py:

- 最小 kernel body。
- guard tid < n。
- global thread id 映射。

coalescing.py:

- consecutive lanes consecutive addresses。

- stride load。
- useful bytes / loaded bytes。

shared_memory.py:

- 32 banks。
- stride conflict。
- broadcast exception。

reduce_kernel.py 和 warp_primitives.py:

- tree reduction。
- warp shuffle reduce。
- ballot mask。

gemm_tiled.py:

- naive global memory traffic。
- tiled shared-memory-style reuse。

capstone_softmax.py:

- numerically stable softmax。
- online softmax。
- 连接 FlashAttention 的 IO-aware 思路。

运行:

`.\.venv\Scripts\python.exe learning\cuda-essentials\src\tests\test_all.py`

21. 和 LLM 系统的连接

CUDA Guide 看似基础，但它直接解释 LLM 系统里的大量现象。

为什么 FlashAttention 快?

减少 global memory traffic

用 shared/register tiling 复用数据

在线 softmax 避免 materialize large score matrix

为什么 decode 小 batch 很难打满 GPU?

GEMV-like shape

low operational intensity

memory traffic dominates

launch overhead and scheduling overhead become visible

为什么 fused kernels 重要?

减少 intermediate tensor 写回 global memory

减少 kernel launch overhead

提高 locality

为什么 CUDA Graphs 对 inference 有意义?

repeated small kernel sequence

CPU submission overhead matters

graph replay reduces repeated launch overhead

为什么 coalescing 重要?

same useful bytes
bad pattern triggers more memory transactions
effective bandwidth collapses

为什么 shared memory bank conflict 会伤性能?

on-chip memory is fast only when parallel access maps well to banks
conflicting accesses serialize

22. 用 AI agent 学 CUDA 的正确方式

不要让 agent 只列 CUDA 概念表。CUDA 必须靠 “形状 + 索引 + 访存 + 同步” 来学。

好的提问:

给定 $n=10000$, $\text{blockDim}=256$, 请计算 gridDim 、最后一个 block 有多少有效 thread。
解释为什么需要 `if (i < n)`。

给定 32 lanes, 每个 lane 读 $\text{base} + \text{lane} * \text{stride} * 4$ 。
 $\text{stride}=1, 2, 8, 32$ 时分别会触发多少 memory sectors?

一个 32×32 shared memory tile 做 transpose, 为什么 column access 会 bank conflict?
把第二维改成 33 后为什么解决?

这个 reduction 能否只用 `__syncthreads`?
什么时候需要 `atomic` 或第二个 kernel?

请用 Roofline 判断这个 kernel 是 memory-bound 还是 compute-bound,
再用 CUDA Guide 的概念列优化动作。

这样学的重点是让 agent 逼你算, 而不是替你背。

23. 新手复习问题

读完后, 建议不用看原文回答:

1. host 和 device 分别是什么?
2. kernel launch 到底启动了什么?
3. grid、block、thread 的层级关系是什么?
4. 为什么普通 CUDA kernel 不能依赖 block 调度顺序?
5. warp 是多少 threads? SIMT 和 SIMD 有什么区别?
6. 为什么 block thread 数最好是 32 的倍数?
7. `threadIdx.x` 为什么通常应该对应连续内存?
8. global memory coalescing 的目标是什么?
9. shared memory 为什么需要 `__syncthreads()`?
10. bank conflict 和 broadcast 有什么区别?
11. occupancy 的分子和分母是什么?
12. stream 内和 stream 间的 ordering 有什么区别?
13. `cudaMemcpyAsync` 为什么需要 pinned host memory 才能真正 overlap?
14. CUDA Graphs 解决什么 overhead?
15. tile programming 和 SIMT 的关系是什么?

24. 一句话收束

CUDA Guide 的核心不是 API 名字，而是执行模型：大量 threads 通过 grid/block/warp 组织起来，在有限 SM 资源上调度，并通过 memory hierarchy、coalescing、shared memory、sync 和 streams 把算法映射到 GPU。你能把这些层级画出来，就开始真正读懂 CUDA kernel。